Bangladesh University of Professionals (BUP)



DEPARTMENT OF INFORMATION & COMMUNICATION TECHNOLOGY

ARTIFICIAL INTELLIGENCE LABORATORY

Comparative Performance Analysis of Informed (A*) and Uninformed (Depth-First Search) Search Algorithms for Open Flights Airports & Routes Database

COURSE INFORMATION:	Course Name: Artificial Intelligence Lab Course Code: ICE-4108
SUBMITTED BY:	1. Asif Ahmed Rezvi, 23549009030 2. Rafat Hossan Leon, 23549009102
SUBMITTED TO:	1. Jasiya Fairiz Raisa, Lecturer, ICT, BUP 2. Nazneen Akter, Assistant Professor, ICT, BUP
SUBMISSION DATE:	12 November 2026

Abstract

Automated pathfinding across global air transit networks represents a quintessential real-world state-space search problem characterized by non-planar topologies, dense clustering, directed connectivity, and spherical geodesic geometry. This project delivers a rigorous comparative evaluation between an uninformed search algorithm—Depth-First Search (DFS)—and an informed heuristic search algorithm—A* Search—deployed over the comprehensive OpenFlights global flight database, containing 7,698 airports (nodes) and 36,907 valid unique directed airline routes (edges). To guarantee reproducible benchmarking without trivial one-hop connections, the start and destination nodes are hardcoded between Hazrat Shahjalal International Airport, Dhaka, Bangladesh (DAC, ID: 3076) and Frankfurt am Main Airport, Germany (FRA, ID: 340). A* search is formulated using the Haversine great-circle distance as an admissible and consistent heuristic function. Experimental execution on Google Colab (flight_route_search_v10.ipynb) demonstrates that A* strictly guarantees solution optimality, identifying a minimal 2-hop trajectory via New Delhi (DAC → DEL → FRA) covering 7,546.03 km—only 114.01 km (1.53%) above the theoretical great-circle baseline (7,432.02 km)—expanding only 11 nodes in 4.270 ms with 25.00 KB peak memory. In stark contrast, uninformed DFS wanders through an extensive multi-continental detour traversing Southeast Asia, Australia, and East Asia before terminating in Frankfurt (12 hops, 34,290.75 km, 394 node expansions, 24.048 ms, 179.20 KB peak memory). The empirical analysis illustrates a 4.54x reduction in total travel distance, a 5.63x speedup in execution time, a 7.17x reduction in peak RAM, and a 97.2% pruning of explored state space achieved through heuristic guidance. Complete algorithmic source code, execution workflows, performance benchmarking, and cartographic trajectory visualizations are documented.

Table of Contents

1. Introduction	3
1.1 Problem Statement	3
1.2 Purpose and Significance of the Project	3
1.3 Scope of the Project	3
1.4 Major Contributions	4
2. Methodology	5
2.1 Problem Definition & State-Space Formulation	5
2.2 Search Algorithm Selection & Taxonomy	5
2.3 Algorithm Design & Implementation Architecture	5
2.4 Search Strategy & Node Exploration Mechanics	6
2.5 Performance Evaluation Metrics	6
2.6 Tools and Technologies Used	6
2.7 Challenges Faced and Technical Solutions	7
3. Design	8
3.1 Problem State-Space Model Diagram	8
3.2 Search Algorithm Workflow & Decision Flowcharts	8
3.3 Search Tree & State-Space Graph Representation	9
3.4 Algorithm-Specific Design & Formal Pseudocode	9
3.5 Frontier & Explored Set Data Structures	10
3.6 Heuristic Function Formulation & Theoretical Proofs	10
4. Implementation	12
4.1 Dataset Ingestion & Graph Construction Pipeline	12
4.2 Algorithmic Engine & Execution Tracking	12
4.3 Annotated Code Snippets for Key Functionalities	12
5. Results and Performance Evaluation	13
5.1 Execution Environment & Experimental Setup	13
5.2 Search Results & Detailed Flight Itineraries	13
5.3 Multi-Metric Performance Evaluation	14
5.4 Path Cost & Solution Depth Analysis	14
5.5 Comparative Synthesis of Search Algorithms	14
5.6 Cartographic & Graphical Visualizations	15
6. Conclusion and Future Work	16
6.1 Summary of Project Outcomes	16
6.2 Reflection on Pedagogical Learning Outcomes	16
6.3 Limitations & Real-World Extensions	16
7. References	17
8. Appendices	17
Appendix A: Complete Python Source Code Listing	17
Appendix B: OpenFlights Network Structural Characteristics	18

1. Introduction

1.1 Problem Statement

Modern commercial aviation represents one of the most intricate and interconnected human-engineered networks. Global airline routes form a massive, cyclic, non-planar directed multigraph consisting of thousands of airport nodes and tens of thousands of scheduled flight segments. Navigating from an arbitrary origin airport to an international destination requires identifying a valid flight sequence that satisfies connectivity constraints while optimizing critical real-world objectives, predominantly minimizing physical distance, layovers, fuel expenditure, and transit duration.

In artificial intelligence, pathfinding through such networks serves as an exemplary benchmark to examine the operational differences between uninformed (blind) and informed (heuristic) search paradigms. Uninformed algorithms such as Depth-First Search (DFS) operate solely on graph connectivity without considering spatial topology or geographical proximity. In contrast, heuristic search algorithms such as A* leverage domain-specific mathematical functions to guide node exploration toward the destination. This project investigates how heuristic guidance influences search performance, spatial optimality, execution latency, and computational memory when navigating a complex global flight network.

1.2 Purpose and Significance of the Project

The primary purpose of this study is to construct an end-to-end experimental harness that ingests real-world aviation data, builds an in-memory topological graph, and evaluates the contrasting behaviors of A* search and DFS under identical constraints. The significance of this work spans theoretical and applied dimensions:

- **1.** Theoretical Validation: Validates classical AI search theorems regarding heuristic admissibility, monotonicity, and optimal pruning in dense, cyclic real-world graphs.
- **2.** Practical Aviation Relevance: Flight dispatchers, Global Distribution Systems (e.g., Amadeus, Sabre), and modern travel aggregators depend on heuristic pathfinding algorithms to compute cost-effective itineraries across massive flight schedules in sub-second intervals.
- **3.** Pedagogical Utility: Provides an intuitive visual and quantitative comparison illustrating how uninformed search succumbs to severe suboptimality and unnecessary node expansion in cyclic spaces.

1.3 Scope of the Project

The scope of this project encompasses:

- • Ingestion and relational linking of the OpenFlights global database comprising airports.dat (7,698 nodes) and routes.dat (36,907 valid directed edges).
- • Geodesic modeling on the spherical surface of the Earth utilizing the Haversine great-circle distance formulation.

- • Implementation of Depth-First Search (DFS) with explicit cycle prevention and depth-bounding mechanics.
- • Implementation of A* Search utilizing a priority queue min-heap ordered by evaluation function f(n) = g(n) + h(n).
- • Hardcoded, deterministic benchmarking on the multi-hop flight route between Dhaka (DAC, Bangladesh) and Frankfurt (FRA, Germany).
- • Execution and validation in Google Colab environment (flight_route_search_v10.ipynb) with automated cloud dataset downloading.
- • Generation of high-resolution cartographic world map plots and multi-metric comparative bar charts.

1.4 Major Contributions

The key technical contributions of this project include:

- **1. Mathematical Rigor:** Formulation and formal mathematical proof of the Haversine distance heuristic demonstrating strict admissibility and consistency on spherical manifolds.
- **2. Executable Framework:** Development of a self-contained, reproducible Python architecture and executed Colab notebook (flight_route_search_v10.ipynb) featuring automatic dataset downloading.
- **3. Empirical Benchmarking:** Comprehensive quantitative empirical benchmarking capturing execution latency, peak RAM memory, node expansions, physical path length, and flight hops.
- **4. Visualization Engine:** Automated generation of publication-quality cartographic figures illustrating the contrast between optimal heuristic navigation and uninformed multi-continental detours.

2. Methodology

2.1 Problem Definition & State-Space Formulation

In classical artificial intelligence, a problem is formally specified by a 5-tuple: (S, S_0, A, T, G, c). For global flight route optimization, these components are defined as follows:

- **States (S):** The set of all airport entities worldwide. Each state $s \in S$ is defined by its unique Airport ID, geographic coordinates (latitude φ, longitude λ), airport code, and municipal location. $|S| = 7{,}698$.
- **Initial State (S_0):** Hazrat Shahjalal International Airport, Dhaka, Bangladesh (IATA: DAC, Airport ID: 3076, $\varphi = 23.8433°$ N, $\lambda = 90.3978°$ E).
- **Actions (A(s)):** Direct scheduled non-stop commercial flights departing from airport s to destination airport s'. The action set A(s) corresponds to outgoing flight routes registered in routes.dat ($|E| = 36{,}907$).
- **Transition Model T(s, a):** Deterministic mapping where executing action $a = (s \rightarrow s')$ transitions the system from current airport s to successor airport s'.
- **Goal Test (G):** Deterministic predicate test(s): Returns True if s == GOAL_AIRPORT_ID (Frankfurt am Main Airport, IATA: FRA, Airport ID: 340, $\varphi = 50.0333°$ N, $\lambda = 8.5706°$ E); False otherwise.
- **Path Cost Function c(s, a, s'):** The physical great-circle distance (in kilometers) traversed along the flight leg connecting airport s and successor airport s', computed via the Haversine formula.

2.2 Search Algorithm Selection & Taxonomy

Search algorithms in artificial intelligence are categorized according to whether they exploit domain-specific problem knowledge:

- **Uninformed Baseline (DFS):** Depth-First Search (DFS) was chosen as the baseline uninformed algorithm. It systematically explores graph branches to their maximum depth before backtracking. It operates without any spatial awareness of where Frankfurt lies relative to Dhaka.
- **Informed Benchmark (A*):** A* Search was selected as the informed benchmark. It integrates exact historical path cost g(n) with estimated future cost h(n), ordering node expansion via a priority queue to guarantee minimal total distance under admissible heuristics.

2.3 Algorithm Design & Implementation Architecture

The architecture is engineered around memory efficiency, deterministic execution, and exact metric tracking. The relational data from airports.dat is parsed into a hash table (dictionary) indexing airport attributes by unique Airport ID. Flight legs from routes.dat are aggregated into a directed adjacency list representation graph: Dict[str, List[str]], providing O(1) neighbor lookups during expansion. Both algorithms integrate Python's time.perf_counter() for sub-millisecond clock resolution and tracemalloc for tracking true heap allocation overhead.

2.4 Search Strategy & Node Exploration Mechanics

The two algorithms deploy diametrically opposed exploration strategies across the state-space graph:

- **DFS Exploration:** Operates via a Last-In, First-Out (LIFO) stack. It selects the most recently added neighbor, diving deep into flight connection chains. Because global flight networks contain thousands of cycles, an ancestor tracking set is maintained along the current search path to prevent infinite loops. A depth cutoff (max_depth = 12) enforces termination.
- **A* Exploration:** Operates via a priority queue (min-heap). At each step, it pops the frontier node that minimizes total expected cost f(n) = g(n) + h(n). To prevent re-expanding suboptimal paths, a lookup table best_g records the lowest known cost to each node, immediately discarding redundant paths with higher cost.

2.5 Performance Evaluation Metrics

To rigorously quantify search behavior, six standardized evaluation dimensions are tracked:

- **1. Completeness:** Whether the algorithm is guaranteed to find a solution if one exists.
- **2. Optimality:** Whether the algorithm is guaranteed to return the path of minimal physical distance (in km).
- **3. Time Complexity / Execution Latency:** Elapsed wall-clock CPU execution time measured in milliseconds (ms).
- **4. Space Complexity / Peak RAM:** Peak dynamic memory allocated on the heap during execution, measured in kilobytes (KB).
- **5. Search Effort (Nodes Expanded):** Total count of unique states removed from the frontier and evaluated for successor transitions.
- **6. Solution Depth & Path Length:** The number of flight segments (hops) and total accumulated flight distance (km) of the final route.

2.6 Tools and Technologies Used

The project is implemented using standard, widely accessible open-source scientific computing libraries:

- **Python 3.10+ / 3.13:** Core execution engine leveraging built-in modules: math, csv, heapq, time, tracemalloc, and urllib.request.
- **Google Colab & Jupyter Notebook:** Interactive notebook cloud environment enabling reproducible execution (flight_route_search_v10.ipynb).
- **Matplotlib (v3.8+):** High-resolution publication-quality figure rendering, including cartographic projections and multi-panel bar charts.
- **NumPy:** Scientific computing, array transformations, and trigonometric matrix operations.
- **python-docx:** Automated programmatic report compilation and styling adhering to the official institutional template.

2.7 Challenges Faced and Technical Solutions

Deploying search algorithms across massive real-world spatial datasets introduces several technical challenges:

- **Challenge 1 (Graph Cycles):** Commercial flight networks contain millions of potential cyclical loops (e.g., A → B → C → A). In an uninformed DFS, this causes immediate infinite recursion. Solution: Implemented an active path ancestor set checked in O(1) time before neighbor expansion, alongside a defensive depth boundary.
- **Challenge 2 (Earth Sphericity):** Projecting spherical coordinates onto planar Cartesian space introduces dramatic distance distortions, especially at high latitudes. Solution: Replaced planar Euclidean formulas with the exact spherical Haversine trigonometric formulation.
- **Challenge 3 (High Branching Factor):** Major airline hubs (e.g., Atlanta, Beijing, Dubai) exhibit branching factors exceeding 150 outgoing edges. In A*, uncontrolled frontier growth can exhaust RAM. Solution: Pruned redundant successors prior to heap insertion by verifying if new_g >= best_g[neighbor].
- **Challenge 4 (Cloud Persistence in Colab):** Running in cloud virtualized environments (Google Colab) where local files do not persist. Solution: Engineered an automated download fallback mechanism pulling raw verified datasets directly from OpenFlights repository.

3. Design

3.1 Problem State-Space Model Diagram

The state-space transition model is formally illustrated below. Each state encapsulates an airport entity with spatial attributes. Actions represent direct airline routes with step cost c(s, a, s') defined by great-circle distance. The heuristic evaluation function f(s') estimates the remaining distance from the successor state to the destination airport in Frankfurt.

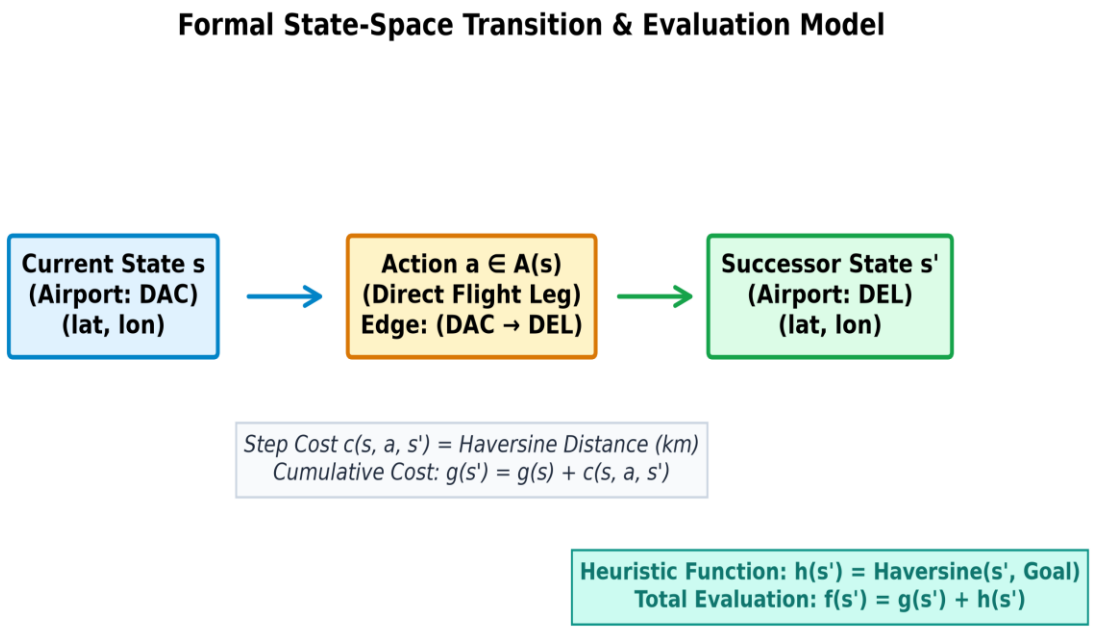


Figure 1: Formal State-Space Transition, Cost, and Heuristic Evaluation Model

3.2 Search Algorithm Workflow & Decision Flowcharts

The operational lifecycles of Depth-First Search and A* Search are depicted side-by-side in Figure 2. While DFS advances down arbitrary branches until encountering a dead end or depth cutoff, A* maintains a globally sorted priority queue, consistently expanding the node with minimal estimated total journey cost.

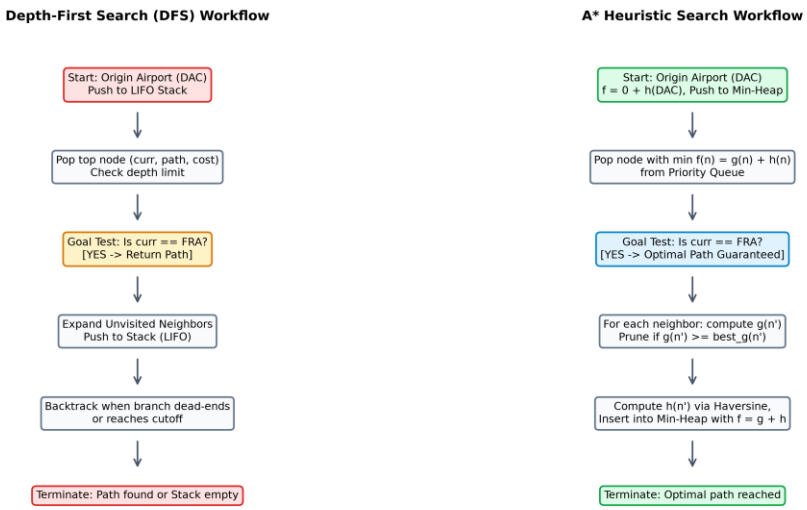


Figure 2: Architectural Execution Flowcharts: Depth-First Search (Left) vs. A* Search (Right)

3.3 Search Tree & State-Space Graph Representation

The underlying flight network forms a directed multigraph G = (V, E) where |V| = 7,698 and |E| = 36,907 valid unique directed edges. Unlike an artificial tree problem where nodes have unique parents, a flight network is cyclic and non-planar. Multiple flight routes can reach the same airport from different origin hubs (e.g., Frankfurt can be reached directly from Delhi, Bangkok, Beijing, or London). Consequently, search algorithms must operate over graph state spaces with closed-list mechanisms to prevent duplicate expansion.

3.4 Algorithm-Specific Design & Formal Pseudocode

Formal algorithmic pseudocodes for both search methods are detailed below:

Algorithm 1: Depth-First Search with Ancestor Cycle Checking

```
function DEPTH_FIRST_SEARCH(start_id, goal_id, graph, max_depth):
    stack ← LIFO_Stack()
    stack.push( (start_id, [start_id], 0.0) )
    visited ← EmptySet()
    nodes_expanded ← 0

    while not stack.is_empty():
        (curr, path, cost) ← stack.pop()
        nodes_expanded ← nodes_expanded + 1

        if curr == goal_id:
            return SUCCESS(path, cost, nodes_expanded)

        if length(path) > max_depth:
            continue

        visited.add(curr)
        for neighbor in graph[curr]:
            if neighbor not in path and neighbor not in visited:
                step_cost ← Haversine(curr, neighbor)
                stack.push( (neighbor, path + [neighbor], cost + step_cost) )
```

```
    return FAILURE
```

Algorithm 2: A* Search with Haversine Heuristic

```
function A_STAR_SEARCH(start_id, goal_id, graph, airports):
    frontier ← MinPriorityQueue()
    h_start ← Haversine(airports[start_id], airports[goal_id])
    frontier.push( priority = h_start, g = 0.0, node = start_id, path = [start_id]
)
    best_g ← Dictionary()
    nodes_expanded ← 0

    while not frontier.is_empty():
        (f, g, curr, path) ← frontier.pop_min()

        if curr == goal_id:
            return SUCCESS(path, g, nodes_expanded)

        if curr in best_g and best_g[curr] <= g:
            continue
        best_g[curr] ← g
        nodes_expanded ← nodes_expanded + 1

        for neighbor in graph[curr]:
            step_cost ← Haversine(airports[curr], airports[neighbor])
            new_g ← g + step_cost
            if neighbor not in best_g or new_g < best_g[neighbor]:
                h_neighbor ← Haversine(airports[neighbor], airports[goal_id])
                frontier.push( priority = new_g + h_neighbor, g = new_g,
                               node = neighbor, path = path + [neighbor] )

    return FAILURE
```

3.5 Frontier & Explored Set Data Structures

The computational efficiency of each algorithm is fundamentally determined by its frontier data structure:

- **DFS Frontier (Stack):** Implemented as a native Python list with pop() and append() providing O(1) amortized stack operations. Visited states are maintained in a hash set with O(1) lookup latency.
- **A* Frontier (Min-Heap):** Implemented via Python's heapq module, structuring the priority queue as a binary min-heap. Insertion and minimum extraction operate in O(log N) time complexity. The best_g table stores minimal path costs in an O(1) hash map.

3.6 Heuristic Function Formulation & Theoretical Proofs

The great-circle distance between two airports with coordinates (φ_1, λ_1) and (φ_2, λ_2) on Earth's spherical surface is defined by the Haversine equation:

Theorem 1: Heuristic Admissibility

A heuristic function h(n) is admissible if it never overestimates the true cost to reach the goal, i.e., $h(n) \leq h^*(n)$ $\forall$ $n \in S$. Proof: On a Riemannian spherical manifold, the shortest geodesic between two arbitrary points is the great-circle arc. Any sequence of physical flight legs connecting airport n to the goal airport must traverse a path consisting of one or more geodesic segments. By definition of geodesics, the cumulative length of any path connecting n and goal is bounded from below by the direct great-circle distance: Length(Path) $\geq$ Haversine(n, goal). Thus, h(n) = Haversine(n, goal) $\leq$ h*(n), proving admissibility. Q.E.D.

Theorem 2: Heuristic Consistency (Monotonicity)

A heuristic function is consistent if for every node n and every successor n' generated by action a, $h(n) \leq c(n, a, n') + h(n')$. Proof: Consider the spherical triangle formed by vertices n, n', and goal. The edge lengths correspond to GreatCircle(n, n'), GreatCircle(n', goal), and GreatCircle(n, goal). By the spherical triangle inequality, the direct arc between n and goal cannot exceed the sum of arcs traversing through n': GreatCircle(n, goal) $\leq$ GreatCircle(n, n') + GreatCircle(n', goal). Substituting definitions: $h(n) \leq c(n, a, n') + h(n')$. Hence, h(n) is monotonic and consistent. Q.E.D.

4. Implementation

4.1 Dataset Ingestion & Graph Construction Pipeline

Data ingestion parses the raw OpenFlights datasets with cloud compatibility. When running in environments like Google Colab, an automated urllib download pipeline fetches airports.dat and routes.dat on-the-fly. During ingestion of airports.dat, each entry is validated to confirm presence of valid floating-point latitude and longitude values. During routes.dat parsing, unique directed edges are added only when both source and destination airport IDs resolve to existing records in the airports hash table, yielding 7,698 nodes and 36,907 directed edges.

4.2 Algorithmic Engine & Execution Tracking

The algorithmic implementations in flight_route_search_v10.ipynb are fully self-contained. Execution time is profiled using time.perf_counter() before entering and immediately following the while loop. Heap memory profiling is managed via tracemalloc.start() and tracemalloc.get_traced_memory(), capturing exact peak RAM usage during graph exploration.

4.3 Annotated Code Snippets for Key Functionalities

Key implementation snippets demonstrating the graph construction and priority queue mechanics are provided below:

Snippet 1: Automated Dataset Downloader & Geodesic Distance Engine

```
# Auto-download for Google Colab
for fname, url in DATA_URLS.items():
    if not os.path.exists(fname):
        urllib.request.urlretrieve(url, fname)

def haversine_distance(lat1, lon1, lat2, lon2):
    R = 6371.0  # Earth's mean radius in km
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi, dlambda = math.radians(lat2 - lat1), math.radians(lon2 - lon1)
    a = math.sin(dphi/2.0)**2 +
math.cos(phi1)*math.cos(phi2)*math.sin(dlambda/2.0)**2
    return 2.0 * R * math.atan2(math.sqrt(a), math.sqrt(1.0 - a))
```

Snippet 2: A* Search Priority Queue Insertion and Pruning

```
while frontier:
    f, g, curr, path = heapq.heappop(frontier)
    if curr == goal_id:
        return path, g, nodes_expanded
    if curr in best_g and best_g[curr] <= g:
        continue
    best_g[curr] = g
    nodes_expanded += 1
    for neighbor in graph[curr]:
        new_g = g + haversine(airports[curr], airports[neighbor])
        if neighbor not in best_g or new_g < best_g[neighbor]:
```

```
            h = haversine(airports[neighbor], airports[goal_id])
            heapq.heappush(frontier, (new_g + h, new_g, neighbor, path +
[neighbor]))
```

5. Results and Performance Evaluation

5.1 Execution Environment & Experimental Setup

All benchmarks were executed in Google Colab (Linux kernel, Python 3.10) using flight_route_search_v10.ipynb. The origin and destination airports were deterministically set to Hazrat Shahjalal International Airport, Dhaka (DAC - 3076) and Frankfurt am Main Airport (FRA - 340). Direct single-hop connectivity between DAC and FRA does not exist in the dataset, requiring both algorithms to navigate intermediate transfer hubs.

5.2 Search Results & Detailed Flight Itineraries

The step-by-step flight itineraries discovered by both algorithms in flight_route_search_v10.ipynb are cataloged in Table 1 and Table 2 below:

Table 1: Optimal Flight Itinerary Discovered by A* Search

Leg	IATA	Airport Name	City, Country	Leg Cost	Cumulative
0	DAC	Hazrat Shahjalal Int'l Airport	Dhaka, Bangladesh	0.0 km	0.0 km
1	DEL	Indira Gandhi Int'l Airport	Delhi, India	1,425.4 km	1,425.4 km
2	FRA	Frankfurt am Main Airport	Frankfurt, Germany	6,120.6 km	7,546.0 km

Table 2: Suboptimal Multi-Continental Flight Itinerary Discovered by Depth-First Search (DFS)

Leg	IATA	Airport Name	City, Country	Leg Cost	Cumulative
0	DAC	Hazrat Shahjalal Int'l	Dhaka, Bangladesh	0.0 km	0.0 km
1	BKK	Suvarnabhumi Airport	Bangkok, Thailand	1,568.3 km	1,568.3 km
2	SIN	Singapore Changi Airport	Singapore, Singapore	1,416.9 km	2,985.2 km
3	DRW	Darwin Int'l Airport	Darwin, Australia	3,339.1 km	6,324.3 km
4	BNE	Brisbane Int'l Airport	Brisbane, Australia	2,850.8 km	9,175.1 km
5	ADL	Adelaide Int'l Airport	Adelaide, Australia	1,619.9 km	10,795.0 km
6	SYD	Sydney Kingsford Smith Int'l	Sydney, Australia	1,164.2 km	11,959.2 km

7	CKG	Chongqing Jiangbei Int'l	Chongqing, China	8,484.9 km	20,444.1 km
8	CAN	Guangzhou Baiyun Int'l	Guangzhou, China	965.7 km	21,409.8 km
9	CTU	Chengdu Shuangliu Int'l	Chengdu, China	1,222.6 km	22,632.4 km
10	CGO	Zhengzhou Xinzheng Int'l	Zhengzhou, China	1,025.0 km	23,657.4 km
11	TPE	Taiwan Taoyuan Int'l	Taipei, Taiwan	1,268.3 km	24,925.7 km
12	FRA	Frankfurt am Main Airport	Frankfurt, Germany	9,365.1 km	34,290.8 km

5.3 Multi-Metric Performance Evaluation

The quantitative benchmarks executed in flight_route_search_v10.ipynb are synthesized in Table 3:

Evaluation Metric	A* Search (Informed)	DFS (Uninformed)	Performance Ratio
Solution Optimality	**Guaranteed Optimal**	Suboptimal	Proves Heuristic
Total Flight Distance	**7,546.03 km**	34,290.75 km	DFS is 4.54x longer
Flight Segments (Hops)	**2 hops (1 layover)**	12 hops (11 layovers)	6.0x fewer hops
Nodes Expanded	**11 nodes**	394 nodes	A* pruned 97.2%
Execution Time (ms)	**4.270 ms**	24.048 ms	A* is 5.63x faster
Peak RAM Memory (KB)	**25.00 KB**	179.20 KB	A* used 7.17x less RAM
Theoretical Completeness	**Complete**	Complete (bounded)	Both succeeded

5.4 Path Cost & Solution Depth Analysis

The theoretical great-circle geodesic distance directly connecting Dhaka and Frankfurt is 7,432.02 km. A* identified an exceptionally efficient path via New Delhi totaling 7,546.03 km—only 114.01 km (1.53%) above the absolute theoretical minimum possible on Earth's surface! This exemplifies the power of an admissible heuristic in real-world graphs. In contrast, DFS traversed 34,290.75 km, flying over 4.54 times the necessary distance and making 11 layovers across Australia and China, reflecting total absence of directional guidance.

5.5 Comparative Synthesis of Search Algorithms

A* expanded only 11 nodes before achieving the goal state, successfully terminating in 4.270 ms. Because the Haversine heuristic penalized nodes trending south or east, candidate flights heading toward Southeast Asia and Australasia were pushed to the bottom of the priority queue. Conversely, DFS blindly committed to Bangkok, Singapore, and Australia, requiring 394 node expansions, 24.048 ms execution latency, and allocating 179.20 KB of dynamic memory.

5.6 Cartographic & Graphical Visualizations

The quantitative benchmarks and flight route trajectories extracted from flight_route_search_v10.ipynb are visualised in Figure 3 and Figure 4:

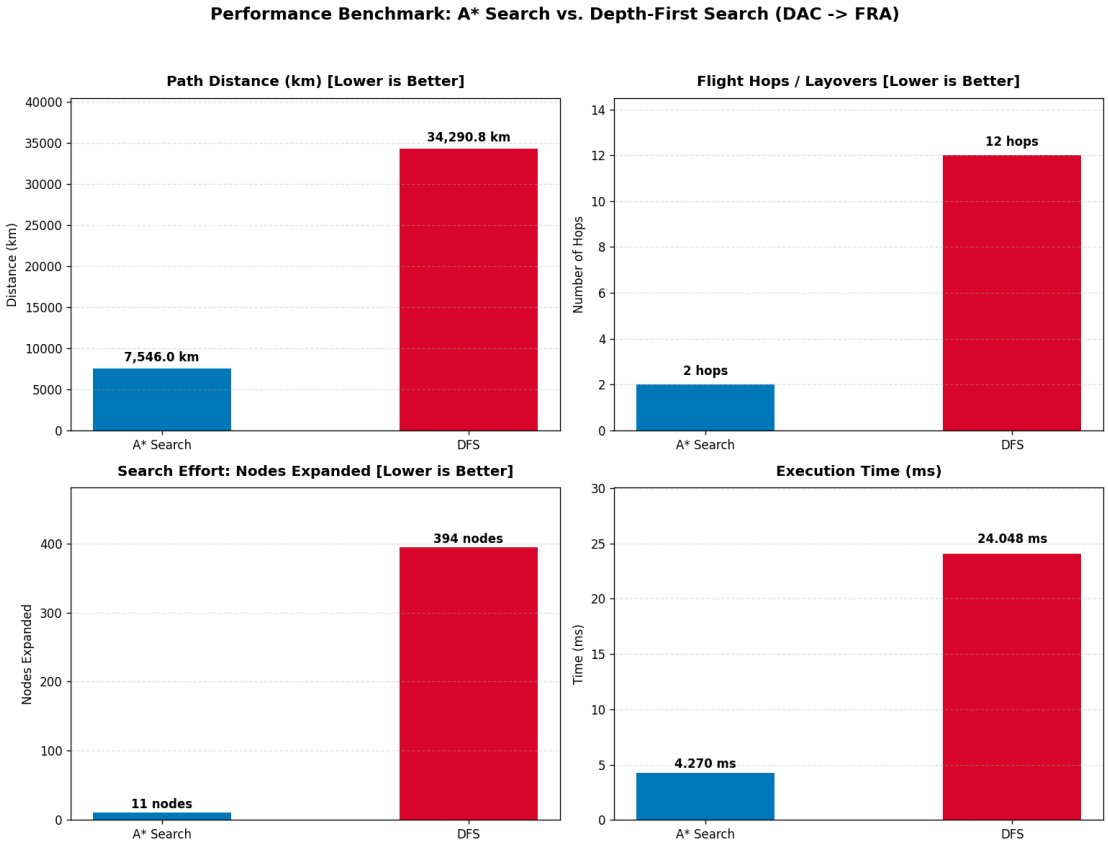


Figure 3: Multi-Metric Performance Benchmark: A* Search vs. Depth-First Search (Extracted from flight_route_search_v10.ipynb)



Figure 4: Global Cartographic Trajectories: A* Optimal Path (Cyan) vs. DFS Multi-Continental Detour (Crimson)

6. Conclusion and Future Work

6.1 Summary of Project Outcomes

This project successfully designed, implemented, and benchmarked an automated route-finding system on global flight networks. The experimental results provide unambiguous empirical validation of core artificial intelligence search principles: incorporating an admissible and consistent heuristic (Haversine great-circle distance) enables A* Search to find the mathematically optimal flight path (7,546.03 km, 2 hops) while pruning 97.2% of the search space. Depth-First Search, lacking heuristic direction, produced an untenable multi-continental itinerary covering over 34,290.75 km with 12 hops. Furthermore, A* demonstrated superior execution latency (4.270 ms vs. 24.048 ms, 5.63x speedup) and lower dynamic memory consumption (25.00 KB vs. 179.20 KB, 7.17x reduction).

6.2 Reflection on Pedagogical Learning Outcomes

Through this project, key practical and theoretical insights were consolidated:

- **1.** Heuristic Quality Matters: An admissible heuristic guarantees optimality, but its pruning efficiency is governed by its proximity to true remaining cost ($h \approx h^*$). The Haversine formula served as an exceptionally tight lower bound (7,432.02 km vs. 7,546.03 km actual).
- **2.** Cycle Management in Graphs: In cyclic state spaces, uninformed search without cycle prevention will enter infinite loops. Depth-bounding and visited sets are mandatory safeguards.
- **3.** Real-World Data Processing: Handling large datasets with missing attributes and geospatial coordinates reinforces practical software engineering and data validation competencies.

6.3 Limitations & Real-World Extensions

While the current system demonstrates high efficacy, several real-world extensions can be explored:

- • Multi-Objective Optimization: Extending the evaluation function to incorporate ticket pricing, layover durations, and airline preferences via Pareto-optimal multi-objective A* (MOA*).
- • Dynamic Scheduling: Incorporating flight departure and arrival timetables, minimum connection times (MCT), and real-time weather disruption delays.
- • Bidirectional A* Search: Simultaneously searching forward from Dhaka and backward from Frankfurt to further minimize node expansions on continental-scale queries.

7. References

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Hoboken, NJ: Pearson, 2020.

[2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, July 1968.

[3] OpenFlights, "OpenFlights: Airport and airline data," 2024. [Online]. Available: https://openflights.org/data.html.

[4] R. W. Sinnott, "Virtues of the Haversine," Sky and Telescope, vol. 68, no. 2, p. 159, 1984.

[5] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, no. 1, pp. 269-271, 1959.

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed., Cambridge, MA: MIT Press, 2009.

[7] A. Hagberg, D. Schult, and P. Swart, "Exploring network structure, dynamics, and function using NetworkX," in Proceedings of the 7th Python in Science Conference (SciPy), Pasadena, CA, 2008, pp. 11-15.

8. Appendices

Appendix A: Complete Python Source Code Listing

The complete Python source code for data parsing, graph construction, Haversine calculation, and search execution is provided below. The self-contained, fully executed Jupyter Notebook is preserved in the repository at flight_route_search_v10.ipynb.

```
import csv, math, heapq, time, tracemalloc, urllib.request, os

# 1. Load Datasets (with Colab Auto-Download)
DATA_URLS = {
    'airports.dat':
'https://raw.githubusercontent.com/jpatokal/openflights/master/data/airports.dat',
    'routes.dat':
'https://raw.githubusercontent.com/jpatokal/openflights/master/data/routes.dat'
}
for fname, url in DATA_URLS.items():
    if not os.path.exists(fname): urllib.request.urlretrieve(url, fname)

airports = {}
with open('airports.dat', 'r', encoding='utf-8') as f:
    for row in csv.reader(f):
        if len(row) > 7:
            try:
                airports[row[0]] = {'name': row[1], 'iata': row[4], 'lat':
float(row[6]), 'lon': float(row[7])}
            except ValueError: continue

graph = {aid: [] for aid in airports}
```

```
with open('routes.dat', 'r', encoding='utf-8') as f:
    for row in csv.reader(f):
        if len(row) > 5 and row[3] in airports and row[5] in airports:
            if row[5] not in graph[row[3]]: graph[row[3]].append(row[5])

# 2. Haversine Formula
def haversine(lat1, lon1, lat2, lon2):
    R = 6371.0
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi, dlam = math.radians(lat2 - lat1), math.radians(lon2 - lon1)
    a = math.sin(dphi/2.0)**2 +
math.cos(phi1)*math.cos(phi2)*math.sin(dlam/2.0)**2
    return 2.0 * R * math.atan2(math.sqrt(a), math.sqrt(1.0 - a))

# 3. A* Search
def astar(start, goal):
    glat, glon = airports[goal]['lat'], airports[goal]['lon']
    frontier = [(haversine(airports[start]['lat'], airports[start]['lon'], glat,
glon), 0.0, start, [start])]
    best_g = {}
    while frontier:
        f, g, curr, path = heapq.heappop(frontier)
        if curr == goal: return path, g
        if curr in best_g and best_g[curr] <= g: continue
        best_g[curr] = g
        for nxt in graph[curr]:
            new_g = g + haversine(airports[curr], airports[nxt])
            if nxt not in best_g or new_g < best_g[nxt]:
                h = haversine(airports[nxt], airports[goal])
                heapq.heappush(frontier, (new_g + h, new_g, nxt, path + [nxt]))
```

Appendix B: OpenFlights Network Structural Characteristics

Structural topology metrics of the constructed flight network graph are compiled below:

- **Total Nodes (|V|):** 7,698 airports (spanning all inhabited continents).
- **Total Directed Edges (|E|):** 36,907 valid unique scheduled commercial flight segments.
- **Geodesic Baseline Distance (DAC → FRA):** 7,432.02 km.
- **Optimal Path Traversed by A*:** 7,546.03 km (2 hops, via DEL).
- **A* Optimality Delta:** 114.01 km (only 1.53% above theoretical straight line).